



武汉大学

WUHAN UNIVERSITY

数据结构与程序设计

(Python语言)

DATA STRUCTURE AND PROGRAM DESIGN IN PYTHON

第3讲 流程控制语句



Python课程组



2025年02月



CONTENTS

目 录

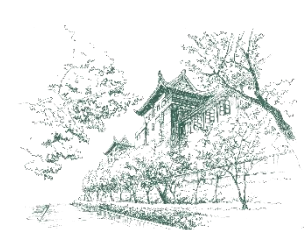
- | | | | |
|---|---------|---|-----------|
| 1 | 算法 | 5 | 循环 |
| 2 | 判断与选择 | 6 | 实践指导 |
| 3 | 条件表达式 | 7 | 程序错误和调试程序 |
| 4 | 多分支选择结构 | | |



武汉大学
WUHAN UNIVERSITY

01

算法



引例

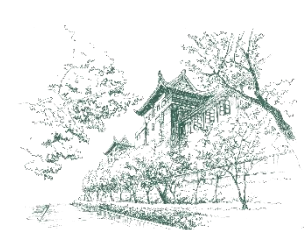


■ 求解最大公约数

- ✿ 最大公约数 (Greatest Common Divisor, 简称GCD) 是指两个或多个整数中最大的能够同时整除它们的数。
- ✿ 换句话说, 它是这些整数公共的因子中最大的一个。

■ 需求分析

- ✿ 编写程序让计算机找出两个正整数的最大公约数。
- ✿ 根据IPO方法, 首先输入两个整数, 然后计算出它们的GCD, 最后输出该GCD。
- ✿ 难点在于设计计算GCD的方法, 也就是程序的关键算法。



引例



■ 设计算法

✱ 采用对所有的可能的解进行“试除”的“笨”办法。

✱ 用自然语言来描述程序的完整算法：

- 第1步，提示输入两个正整数，分别赋值给变量a和b。
- 第2步，用a和b中较小的数作为候选GCD赋值给变量gcd，然后用gcd去除较大数，如果能整除，那么gcd就是解，转到第5步；否则，进入下一步。
- 第3步，把gcd的值减一，得到新的候选GCD。
- 第4步，用候选GCD去除两个数，如果都能整除，则候选GCD就是解，转到第5步；否则，转到第3步。
- 第5步，输出找到的GCD（即变量gcd当前引用的值），算法结束。



算法的概念与特征



武汉大学
WUHAN UNIVERSITY

- 算法是解决特定类型问题的明确、有限的步骤序列，其通过接收输入、执行有效操作，在有限时间内生成预期输出。
- 算法被称为程序的“灵魂”，具有以下特征：
 - ✿ 输入：接受零个或多个输入。
 - ✿ 输出：至少产生一个输出。
 - ✿ 有穷性：必须在有限的步骤内执行完毕。
 - ✿ 确定性：每个步骤必须明确且无歧义。
 - ✿ 可行性：每个步骤都可以通过基本操作实现。



算法的概念与特征



武汉大学
WUHAN UNIVERSITY

■ 软件的核心是算法

- ✿ 排序算法
- ✿ 查找算法
- ✿ 图搜索算法
- ✿ 整数因子分解算法
- ✿ 图像压缩算法
- ✿ 随机数生成算法
- ✿ 加密算法
- ✿

算法是程序的灵魂。

Algorithm is the spirit of a program.



算法的表示



武汉大学
WUHAN UNIVERSITY

■ 算法的表示方法

- ✿ 自然语言 给人看的
- ✿ 伪代码 给人看的
- ✿ 流程图 给人看的
- ✿ 计算机语言 目的是让机器可以执行算法

■ 举例

✿ 问题

- 计算 $1+2+3+\dots+100$

✿ 数据结构

- 定义整型变量 i 保存加数
- 定义整型变量 s 保存每次累加的和



算法的自然语言表示



武汉大学
WUHAN UNIVERSITY

问题

✱ 计算 $1+2+3+\dots+100$

算法的自然语言表示

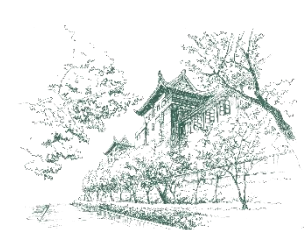
步骤1: $i=1, s=0$ 。

步骤2: 如果 i 小于或等于100, 顺序执行步骤3; 否则, 执行步骤5。

步骤3: s 加上 i , 相加后的值仍放在 s 中, 即: $s = s+i$ 。

步骤4: 使 i 的值增一得到下一个加数, 即 $i=i+1$; 执行步骤2。

步骤5: 变量 s 中的值就是要得到的结果; 输出结果, 算法结束。



算法的伪代码表示



问题

✱ 计算 $1+2+3+\dots+100$

用伪代码表示算法

```
i = 1
s = 0

while i<=100 do
    s = s + i
    i = i + 1
end while

print s
```

- 非正式的语言
- 与日常用语类似
- 帮助程序员在写程序之前“设想出”程序
 - 很容易被转换成正式的程序
 - 一般只包括可执行语句

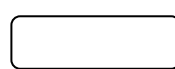
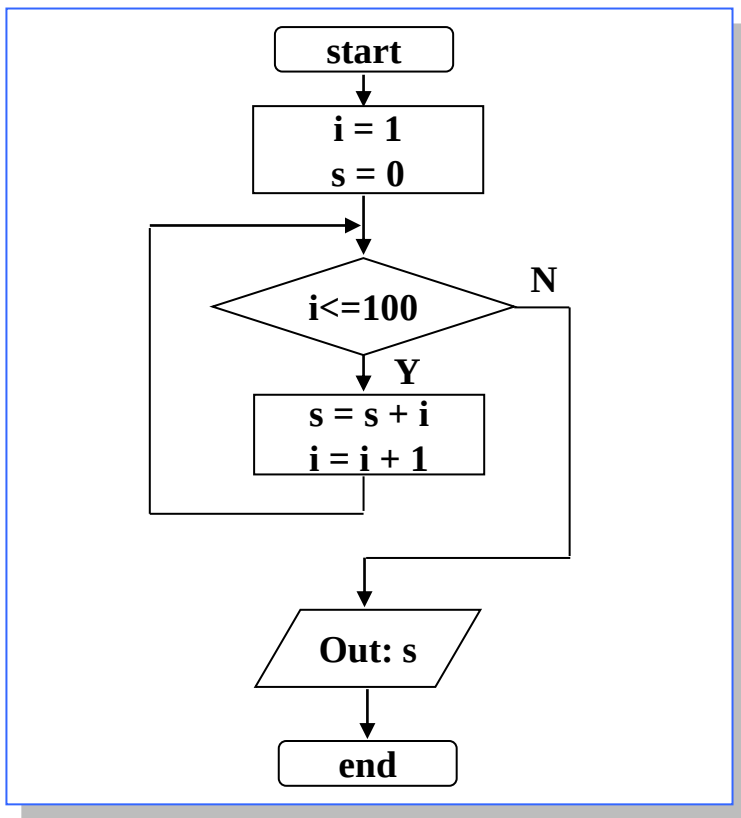


算法的流程图表示

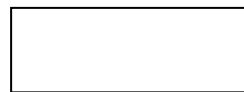


武汉大学
WUHAN UNIVERSITY

用流程图表示算法



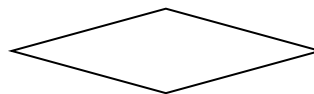
程序的开始和结束



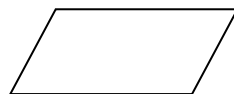
动作



流程线



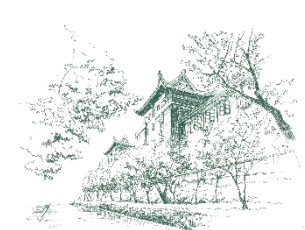
判断



输入/输出



连接



算法的计算机语言表示



武汉大学
WUHAN UNIVERSITY

■ 用Python语言表示（实现）算法

```
i = 1                                # 数据结构
s = 0

while i <= 100:                       # 累加100次
    s = s + i
    i = i + 1

print("1+2+3+...+100 =", s);          # 输出结果
```

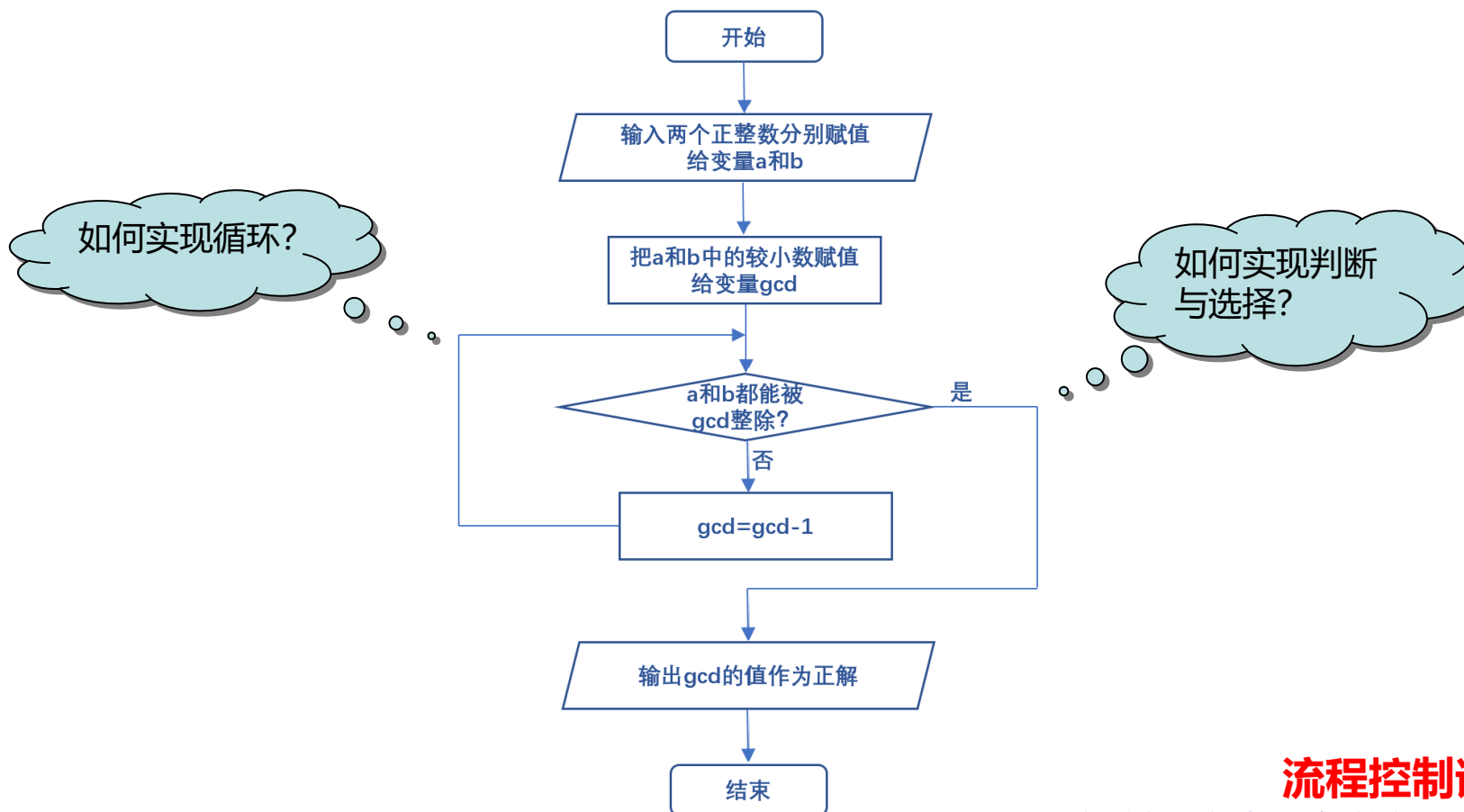



引例



武汉大学
WUHAN UNIVERSITY

最大公约数算法的流程图表示



流程控制语句
控制程序中语句的执行顺序



武汉大学
WUHAN UNIVERSITY

02

判断和选择



主要内容



武汉大学
WUHAN UNIVERSITY

- 判断（关系运算和逻辑运算）

- 选择结构

 - ✱ if

 - ✱ if-else

- 嵌套选择结构



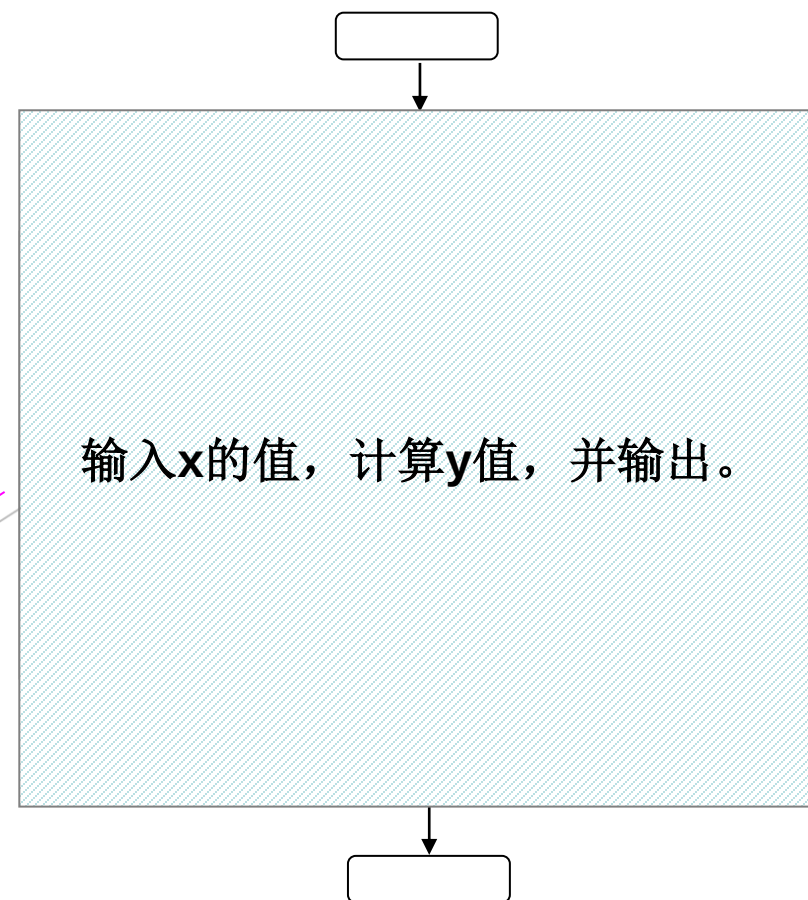
引例：程序流程控制

引例

✿ 问题：根据输入的x值，计算y值。

$$y = \begin{cases} x^2 + 1 & (x \leq 2.5) \\ x^2 - 1 & (x > 2.5) \end{cases}$$

有了合适的数据类型和数据结构之后，还要依赖于选择和循环结构来实现特定的业务逻辑。





引例：程序流程控制



武汉大学
WUHAN UNIVERSITY

引例

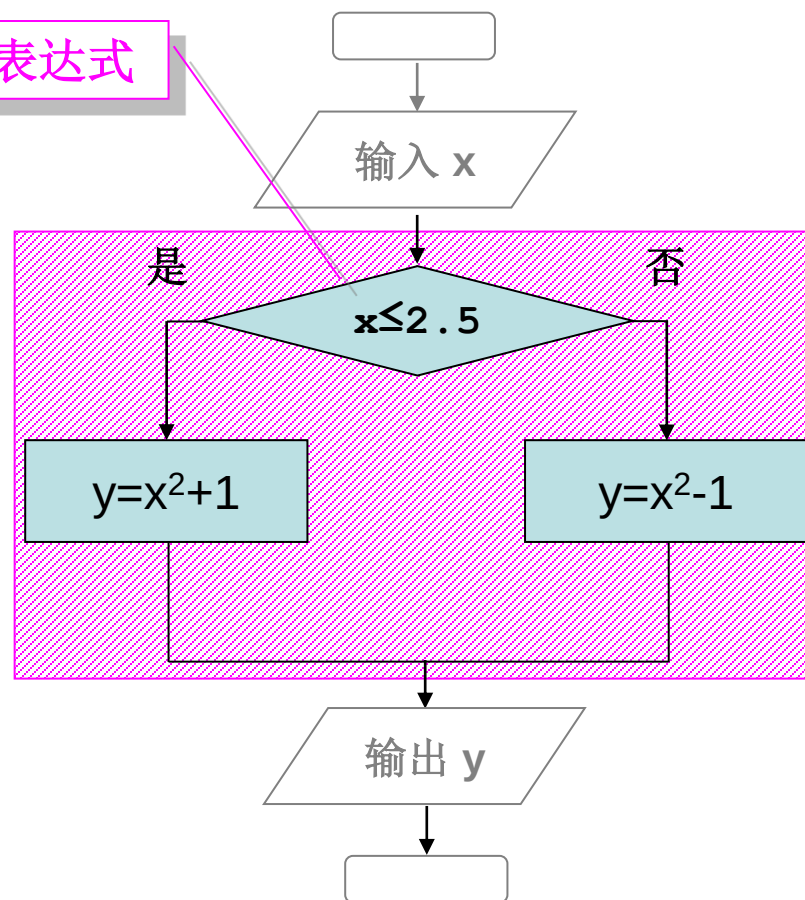
源代码

```
x = float(input("x = "))
```

```
if x<=2.5:  
    y = x**2+1  
else:  
    y = x**2-1
```

```
print("y =", y)
```

条件检测表达式





条件检测表达式



■ 根据条件检测表达式的值做出判断

- ✱ 在选择和循环结构中，条件检测表达式的值只要不是False、0（或0.0、0j等）、空值None、空列表、空元组、空集合、空字典、空字符串、空range对象或其他空迭代对象，Python解释器均认为与True等价。

- ✱ 例如：123、"abc"、(1,2)均为True

■ 构造条件检测表达式最常使用的运算

- ✱ 关系运算
- ✱ 逻辑运算



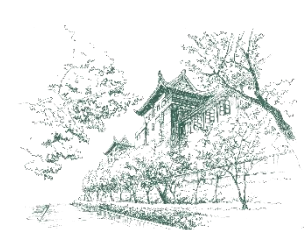
关系运算符

< <= > >= == !=

✱ Python中的关系运算符可以连续使用，这样不仅可以减少代码量，也比较符合人类的思维方式。

```
>>> 12 > 7
True
>>> 34.5 < 6.78
False
>>> 'abc' >= 'abc'
True
>>> 'abc' < 'abcd'
True
>>> 12/3 == 4
True
>>> 13%2 != 0
True
```

```
>>> print(1<2<3)    #等价于 "1<2 并且 2<3"
True
>>> print(1<2>3)
False
>>> print(1<3>2)
True
```

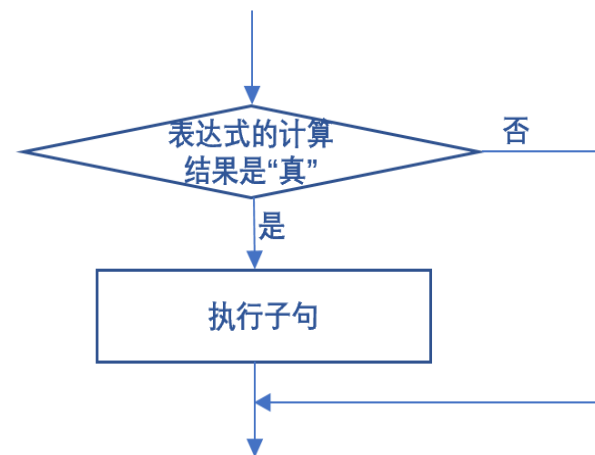


单分支选择结构



单分支选择结构

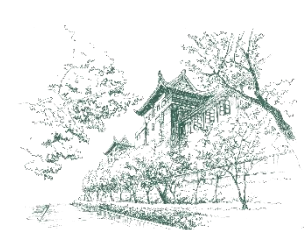
if 表达式:
子句



```
"""判断输入的正整数是否能被3整除，若能，则输出它是3的几倍数。"""  
number = int(input("输入一个正整数: "))  
if number % 3 == 0:  
    print(f"{number} 是 3 的 {number//3} 倍数。")
```

输入一个正整数: 12
12 是 3 的 4 倍数。

输入一个正整数: 14



双分支选择结构



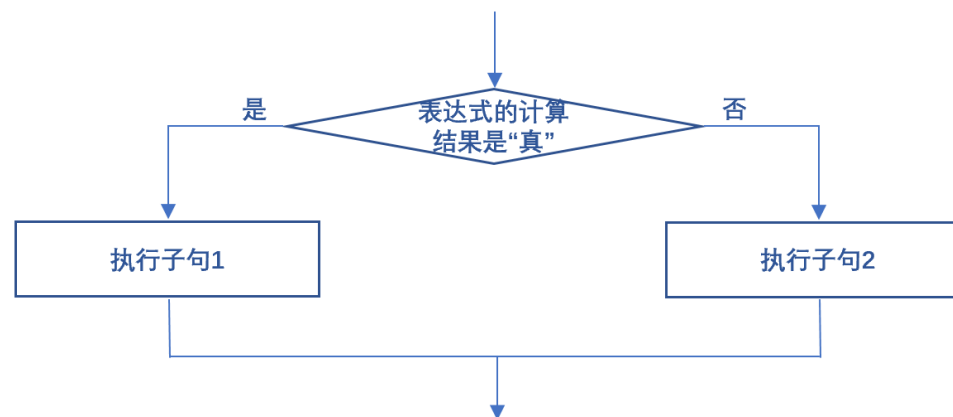
■ 双分支选择结构

if 表达式:

子句1

else:

子句2



"""判断输入的正整数是否能被3整除，若能，则输出它是3的几倍数。"""

```
number = int(input("输入一个正整数: "))
```

```
if number % 3 == 0:
```

```
    print(f"{number} 是 3 的 {number//3} 倍数。")
```

```
else:
```

```
    print(f"{number} 不是 3 的倍数。")
```

输入一个正整数: 14

14 不是 3 的倍数。



嵌套选择结构



嵌套选择结构

if 表达式1:

语句块1

if 表达式2:

语句块2

else:

语句块3

else:

if 表达式4:

语句块4

```
1 | if 表达式 1:
  |   语句块 1
  |   if 表达式 2:
  |       2 | 3 | 语句块 2
  |   else:
  |       3 | 语句块 3
  |   else:
  |       2 | if 表达式 4:
  |           3 | 语句块 4
```

"""在输入的3个整数中找出最大数。"""

```
a,b,c = eval(input("输入3个整数，用逗号作为分隔符："))
```

```
if a>=b:
    if a>=c:
        print(f"最大数是 {a}")
    else:
        print(f"最大数是 {c}")
else:
    if b>=c:
        print(f"最大数是 {b}")
    else:
        print(f"最大数是 {c}")
```

注意：缩进必须要正确并且一致。



嵌套选择结构



思考:

下面三个if语句的区别

```
if (i>0):  
    if (j>0): n=1  
else: n=2
```

```
if (i>0):  
    if (j>0): n=1  
    else: n=2
```

```
if (i>0): n=1  
else:  
    if (j>0): n=2
```



关系运算的不足



■ 关系表达式只能表达一些简单的判断

`grade >= 60`

`total > 1000`

- ✱ 每个判断只是对一个条件进行检测。
- ✱ 如果要对多个条件进行检测就要在多个并列的if语句或嵌套的if或if-else语句中进行检测。
- ✱ 例1： `grade >= 60` 并且 `gender == '男'`

```
if grade >= 60:  
    if gender == '男':  
        count += 1
```

- ✱ 例2： `x < -10` 或者 `x > 0`

```
if x < -10: print('无效数据')  
if x > 0: print('无效数据')
```




逻辑运算

✱ 通过逻辑运算符把简单的条件组合起来，能够形成更加复杂的判断。

✱ 例1：“`grade >= 60`并且`gender == '男'`”的逻辑表达式

`(grade >= 60) and (gender == '男')`

✱ 例2：`x < -10`或者`x > 0`的逻辑表达式

`(x < -10) or (x > 0)`

逻辑运算符

逻辑与：`and`

逻辑或：`or`

逻辑非：`not`



逻辑表达式

[<表达式1>] <逻辑运算符> <表达式2>

✱ 表达式1和表达式2可以是任何表达式。

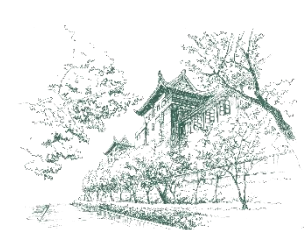
✱ 举例

```
(x>5) or (x<-5)
not (a<b) and (m!=n)
(a+b>c) and (a+c>b) and (b+c>a)
```

逻辑表达式的值

True: 真

False: 假

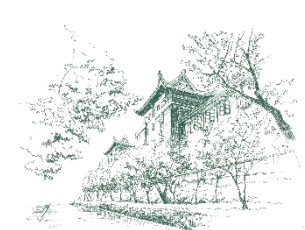


逻辑运算规则

✱ 把参与逻辑运算的操作数据看作bool值

bool(a)	bool(b)	a and b	a or b	not a
True	True	b True	a True	False
True	False	b False	a True	False
False	True	a False	b True	True
False	False	a False	b False	True

其中，a 和 b 是可以转换为布尔类型值的任何对象。

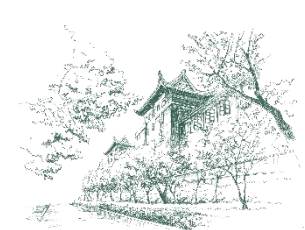


逻辑表达式举例

```
>>> x = 1.2
>>> y = 3.4
>>> z = 5.6
>>> x < y and x < z
True
>>> gender = '女'
>>> age = 65
>>> gender == '女' and 60 <= age < 70
True
>>> not gender == '女'
False
>>> not gender == '男'
True
>>> x > 0 or x < 2
True
>>> x > 0 or x > y
True
```

这时，结果是最后一个
被计算的表达式的值

```
>>> x or y or z
1.2
>>> x and y and z
5.6
```



2.5 逻辑运算

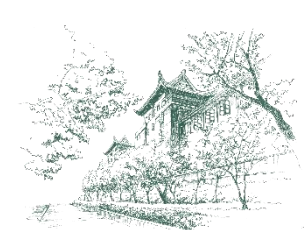
■ 逻辑运算的“惰性”（逻辑短运算，short-circuit evaluation）

✱ 逻辑运算符and和or具有短路求值或惰性求值的特点

- 可能不会对所有表达式进行求值，而是只计算必须计算的表达式的值。
- 例如： `gender=='F' and age>=60`
 - 若`gender`不等于'F'，则表达式的结果为`False`，而不去检查`age`的值

✱ 优化代码

- 在设计包含多个条件的条件表达式时，如果能够大概预测不同条件失败的概率，并将多个条件根据“`and`”和“`or`”运算符的短路求值特性来组织顺序，可以提高程序的运行效率。



逻辑表达式的应用



武汉大学
WUHAN UNIVERSITY

案例：闰年判断

✿ 输入一个年份，判断并输出其是闰年还是平年。

```
"""判断某年是否为闰年"""  
year = int(input("输入一个年份（正整数）："))  
  
if year % 400 == 0 or (year % 4 == 0 and year % 100 != 0):  
    print(f"{year} 年是闰年。")  
else:  
    print(f"{year} 年是平年。")
```

输入一个年份（正整数）：2024
2024 年是闰年。

输入一个年份（正整数）：2025
2025 年是平年。



武汉大学
WUHAN UNIVERSITY

03

条件表达式



■ 条件运算符

✿ 一个三元运算符，连接3个操作数构造条件表达式

值1 if 表达式 else 值2

✿ 计算规则是：

➤ 整个表达式的计算结果是值1或者值2，取决于if和else间表达式的计算结果被转换为布尔值时是真还是假。如果是真，表达式的结果为值1，反之，则为值2。

✿ 例如：

```
>>> x = 123
>>> y = x+1 if x>0 else x-1    #赋值运算符优先级最低
>>> y
124
>>> print(f"{x}是{'奇数' if x%2 else '偶数'}")
123是奇数
```



武汉大学
WUHAN UNIVERSITY

04

多分支选择结构



多分支选择结构(if-elif-...-else语句)



■ if-elif-...-else语句

if 表达式1:

子句1

elif 表达式2:

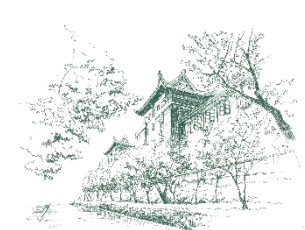
子句2

.....

else:

子句n

其中，关键字elif是else if的缩写。



多分支选择结构示例



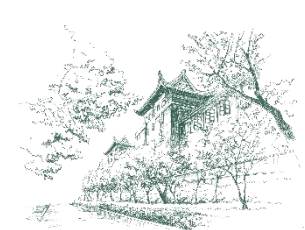
案例：

✿ 使用多分支选择结构将成绩从百分制变换到等级制。

```
score = int(input("输入考试成绩（百分制）："))

print("换成等级：", end='')
if score >= 90:
    print('A')
elif score >= 80:
    print('B')
elif score >= 70:
    print('C')
elif score >= 60:
    print('D')
else:
    print('E')
```

注意这里隐含的情况：
成绩小于90



多分支选择结构示例



案例：判断坐标点象限

✿ 已知坐标点(x,y)，判断其所在的象限。

```
x = int(input("请输入x坐标: "))
y = int(input("请输入y坐标: "))

if (x == 0 and y == 0): print("位于原点")
elif (x == 0): print("位于y轴")
elif (y == 0): print("位于x轴")
elif (x > 0 and y > 0): print("位于第一象限")
elif (x < 0 and y > 0): print("位于第二象限")
elif (x < 0 and y < 0): print("位于第三象限")
else: print("位于第四象限")
```




多分支选择结构示例



■ 练习：出租车计价程序

✿ 下面是某城市出租车收费标准：

- 起步里程为3公里，起步费10元；
- 超起步里程后10公里以内，每公里2元；
- 超过10公里以上的部分加收50%的回空补贴费，即每公里3元。

✿ 请根据该标准编写出租车计费程序，输入总里程数，输出应付费用。



多分支选择结构示例



出租车计价程序的参考实现

```
'''
    出租车计价程序
    某城市...
'''
import math

rawdis = float(input('输入总里程数（公里）： '))

dis = math.ceil(rawdis) # 不足1公里的算1公里

fare = 10
if dis>10:
    fare += (dis-10)*3 + 7*2
elif dis>3:
    fare += (dis-3)*2

print('总里程数{:.1f}公里，应付{:.2f}元'.format(rawdis, fare))
```

输入总里程数（公里）： 3
总里程数3.0公里，应付10.00元

输入总里程数（公里）： 3.1
总里程数3.1公里，应付12.00元

输入总里程数（公里）： 4.6
总里程数4.6公里，应付14.00元

输入总里程数（公里）： 12.1
总里程数12.1公里，应付33.00元



多分支选择结构（match语句）

match语句构造的多分支选择结构

match 表达式:

case 字面值1:

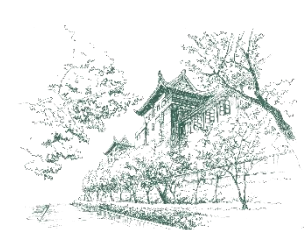
子句1

case 字面值2:

子句2

.....

- ✿ Python在3.10版中新增加了match语句。
- ✿ 计算match后面表达式的值，检测该值与哪一个case后面的字面值相等（匹配），则执行该case下的子句。

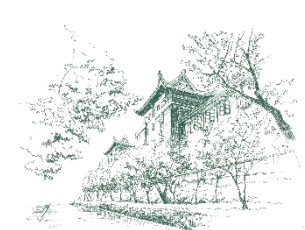


多分支选择结构示例



案例：购票程序

```
"""根据用户输入的选项执行相应的操作"""  
print("""  
本次列车提供以下3种类型的座位：  
[1] 商务座  
[2] 一等座  
[3] 二等座  
""")  
site_type = int(input("请输入您的选择（1、2、3）："))  
  
match site_type:  
    case 1:  
        print("您选了商务座，正在为您准备。")  
    case 2:  
        print("您选了一等座，正在为您准备。")  
    case 3:  
        print("您选了二等座，正在为您准备。")  
    case _:  
        print("您输入了错误的数字。")
```



小结



■ 构造选择结构的流程控制语句

- ✿ if

- ✿ if-else

- ✿ if-elif-else

- ✿ match-case

■ 关系运算和逻辑运算

■ 条件表达式

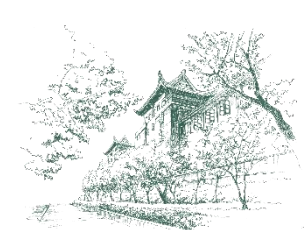
■ 智能的一个体现方面是根据实际环境来调节反应的能力，所以构造选择结构的语句是开发具有智能行为程序的基础。



武汉大学
WUHAN UNIVERSITY

05

循环



循环结构



武汉大学
WUHAN UNIVERSITY

- while语句
- for语句
- range对象
- break语句
- continue语句
- 程序设计举例



引例



■ 引例：购物

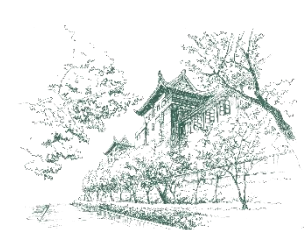
...

while 购物单上还有商品

购买一件商品，并且把它从清单上划掉

...

- ✿ 如果条件“购物单上还有商品”成立时，就执行动作“购买一件商品，并且把它从清单上划掉”；
- ✿ 如果该条件一直成立，这个动作就会重复执行；
- ✿ 最终（该买的商品都买到），即该条件不成立，循环过程就会终止，程序将执行这个循环结构之后的第一条语句。



两种循环语句



- Python主要有for循环和while循环两种形式的循环结构。
 - ✱ while循环一般用于循环次数难以提前确定的情况，当然也可以用于循环次数确定的情况。
 - ✱ for循环一般用于循环次数可以提前确定的情况，尤其适用于枚举或遍历序列或迭代对象中元素的场合。
- 多个循环可以嵌套使用，并且还经常和选择结构嵌套使用来实现复杂的业务逻辑。



while语句

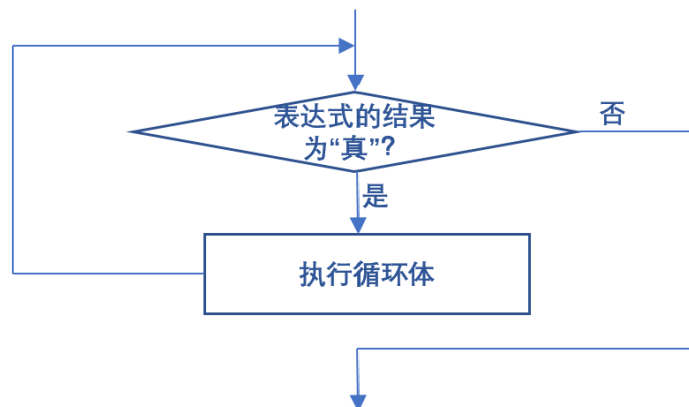


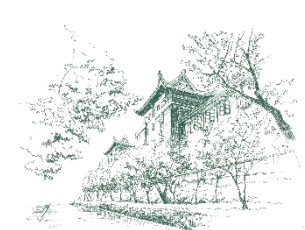
while语句的一般写法

while 表达式:
循环体

说明:

➤ while语句构造的是一个当型循环，即当某个条件满足时重复执行一组操作，直到条件不满足。





while循环的应用示例



案例：求解最大公约数

```
"""输入两个正整数，找出它们的最大公约数（GCD）"""  
  
# 输入两个正整数分别赋值给变量a和b  
a, b = eval(input("输入两个正整数，用逗号隔开："))  
  
# a和b中较小的数赋值给变量gcd，作为第一个候选GCD  
gcd = a if a < b else b  
  
# 用“试除”法找GCD  
while a % gcd != 0 or b % gcd != 0:    # 检测当前的gcd能否把a和b整除，满足则找到了GCD，  
    gcd = gcd - 1                    # 否则还没找到GCD  
    # 没找到GCD，计算下一个候选GCD  
  
# 当前的gcd就是要找的GCD，输出该数  
print(f"这两个数的最大公约数是 {gcd}")
```

```
输入两个正整数，用逗号隔开：12,56  
这两个数的最大公约数是 4
```



while循环的应用示例



案例：百鸡百钱问题

我国古代数学家张丘建在《算经》一书中提出的数学问题：“鸡翁一值钱五，鸡母一值钱三，鸡雏三值钱一。百钱买百鸡，问鸡翁、鸡母、鸡雏各几何？”

```
"""用枚举法求解百鸡百钱问题"""
print("所有解：")
print("公鸡 母鸡 小鸡")

num_cock = 0 # 公鸡数量
while num_cock <= 100 // 5:
    num_hen = 0 # 母鸡数量
    while num_hen <= 100 // 3:
        num_chick = 100 - num_cock - num_hen # 小鸡的数量
        if num_chick >= 0 and num_chick % 3 == 0 \
            and 5 * num_cock + 3 * num_hen + num_chick // 3 == 100:
            print(f"{num_cock:4d} {num_hen:4d} {num_chick:4d}")

        num_hen += 1 # 计算母鸡数量的下一个候选值
    num_cock += 1 # 计算公鸡数量的下一个候选值
```

所有解：

公鸡	母鸡	小鸡
0	25	75
4	18	78
8	11	81
12	4	84



5.1 while循环的应用



■ 枚举法（穷举法）

- ✱ 枚举法是一种解决问题的方法，通过尝试所有可能的情况来找到问题的解。
- ✱ 枚举法还可以用来求解排列组合问题、旅行商问题、图着色问题、数独问题、单词搜索问题、0-1背包问题、最大子序列问题等等。



while循环的应用示例



案例：计数器控制的确定循环

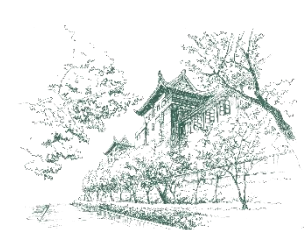
✿ 编写程序，输入某班10个学生某次考试的成绩，输出平均分。

```
s = 0.0

i=0
while i<10:
    score = float(input('Input score:'))
    s += score
    i += 1

print('average is', s/i)
```

```
Input score:1
Input score:2
Input score:3
Input score:4
Input score:5
Input score:6
Input score:7
Input score:8
Input score:9
Input score:10
average is 5.5
```



while循环的应用示例



案例：标记控制的不确定循环

✿ 编写程序，输入某班若干个学生某次考试的成绩，输出平均分。

```
s = 0.0
i=0

score = float(input('Input score (invalid one to end):'))

while 0<=score<=100:
    s += score
    i += 1
    score = float(input('Input score (invalid one to end):'))

if i>0:
    print('average is', s/i)
else:
    print('no valid score')
```

定义标记值：输入成绩时输入无效值表示结束输入



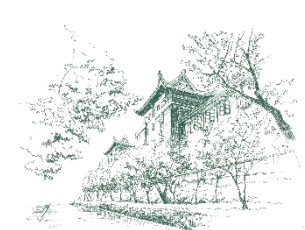
while循环的应用示例



练习：取款机密码验证

✿ 在取款机上取款时需要输入6位银行卡密码。下面模拟一个简单的取款机（密码只有1位数字），

- 每次要求用户输入1位数字密码，密码正确输出“密码输入正确，正进入系统...”；
- 如果输入错误，输出“密码输入错误，您已经输入*次”，密码连续输入错误5次后输出“密码输入错误，您还可以输入1次密码”，密码连续输入错误6次后输出“您的卡将被锁死，请联系发卡行联系”。



5.1 while循环的应用



■ 取款机密码验证的参考实现

```
password = 8    # 密码
trymax = 6      # 输入密码的最大次数
trycount = 0    # 输入密码的次数

while trycount < trymax:
    inpwd = int(input('请输入1位数密码: '))
    if inpwd == password:
        print('密码输入正确, 正进入系统...')
        trycount = 99    # 为了结束循环
    else:
        trycount += 1
        if trymax - trycount == 1:
            print('密码输入错误, 您还可以输入1次密码')
        elif trycount < trymax:
            print('密码输入错误, 您已经输入{}次'.format(trycount))
        else:
            print('您的卡将被锁死, 请和发卡行联系')
```



while-else语句

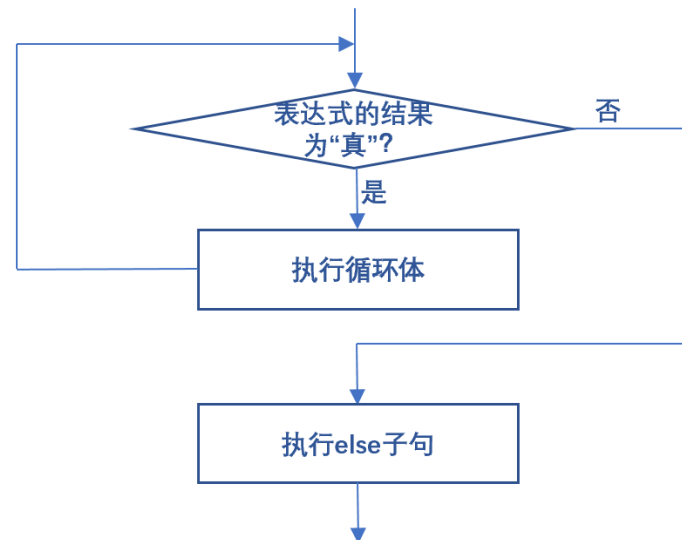


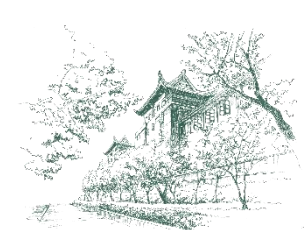
while-else语句

while 条件检测表达式:
 循环体
else:
 子句

※ 注意:

- 如果循环因为条件检测表达式不成立而自然结束时则执行**else**子句;
- 如果循环是因为执行了**break**语句而导致循环提前结束则不会执行**else**中的语句。





break语句



■ 使用说明:

- ✿ 用在for循环和while循环内部。
- ✿ 一旦break语句被执行，将使得break语句所属层次的循环提前结束。
- ✿ 执行break语句后，不会执行for和while的else子句。



break语句应用示例



案例

✿ 编写程序判断输入的大于1的整数是否是素数。

➤ 素数，又称质数，是指一个大于1的自然数，除了1和它本身外，不能被其他自然数整除的数。换句话说，素数是只能被1和自身整除的正整数。

✿ 设计：用枚举法判断一个大于1的自然数是否是素数。

```
"""输入一个大于1的整数，判断它是否是素数"""
n = int(input("输入一个大于1的整数: "))

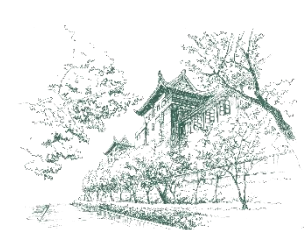
factor = 2
while factor < n:
    if n % factor == 0:
        print(f"它被{factor}整除，所以它不是素数。")
        break
    factor += 1
else:
    print("它是素数。")
```



continue语句



- 在循环语句的循环体中还可以使用continue语句，用来结束本轮循环，直接开始下一轮循环，即执行循环条件表达式。
- 因此，continue语句也被称为继续语句。



continue语句应用示例



案例

✿ 输入符合要求的数据。

```
"""输入5个大于6的偶数"""  
count = 0  # 统计满足条件的偶数个数  
  
while count < 5:  
    value = int(input("输入一个大于6的偶数: "))  
    if value <= 6 or value % 2 != 0:  
        print("不符合要求。")  
        continue  # 结束本次循环  
  
    count += 1  
    print(f"很好。已输入 {count} 个符合要求的数。")
```



break与continue语句应用示例



案例

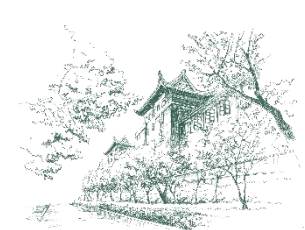
✿ 输出小于100的最大素数。

```
n = 100
while n > 1:
    if n%2 == 0:      # 优化
        n -= 1
        continue

    i = 3
    while i <= int(n**0.5):    # 进一步优化
        if n%i == 0:
            break # 结束内循环

        i += 2
    else:
        print(n)
        break # 结束外循环

    n -= 1
```



更好的最大公约数算法



更相减损法

✿ 中国古代数学著作《九章算术》中的方法。

➤ 将两个数**a**和**b**相减，如果差**c**等于0，那么最大公约数为**a**，如果**c**不等于0，则将**b**和**c**中的较大数赋值给**a**，较小数赋值给**b**，继续相减直到差等于0，此时**a**的值就是最大公约数。

✿ 例如，用更相减损法求15和21的最大公约数的过程：

$$\begin{array}{rcl} 21 & - & 15 = 6 \\ 15 & - & 6 = 9 \\ 9 & - & 6 = 3 \\ 6 & - & 3 = 3 \\ 3 & - & 3 = 0 \end{array}$$

➤ 因此，15和21的最大公约数是3。



更好的最大公约数算法



更相减损法

```
"""用更相减损法求解最大公约数"""
```

```
a, b = eval(input("输入两个正整数，用逗号隔开: "))
```

```
while a - b != 0:
```

```
    if a < b: a, b = b, a
```

```
    a, b = b, a-b
```

```
print(f"这两个数的最大公约数是 {a}")
```



更好的最大公约数算法



辗转相除法

✿ 古希腊数学家欧几里得提出。

➤ 将两个数 a 和 b 相除，大数 a 作为被除数，小数 b 作为除数，若二者余数不为0，则将小数作为被除数，余数作为除数，...直到余数为0，小数 b 就是最大公约数。

✿ 例如，用辗转相除法求15和21的最大公约数的过程：

$$\begin{aligned} 21 &\% 15 = 6 \\ 15 &\% 6 = 3 \\ 6 &\% 3 = 0 \end{aligned}$$

➤ 得到最大公约数3。

你来动手试一试



程序设计示例 (迭代法)



武汉大学
WUHAN UNIVERSITY

问题

✿ 用如下近似公式求自然对数的底数e的值，直到最后一项的绝对值小于 10^{-6} 为止。

$$e \approx 1 + \frac{1}{1!} + \frac{1}{2!} + \dots + \frac{1}{n!}$$

```
e = ai = i = 1
while ai >= 1e-6:
    e += ai    # e是迭代变量

    i += 1
    ai /= i    # ai也是迭代变量

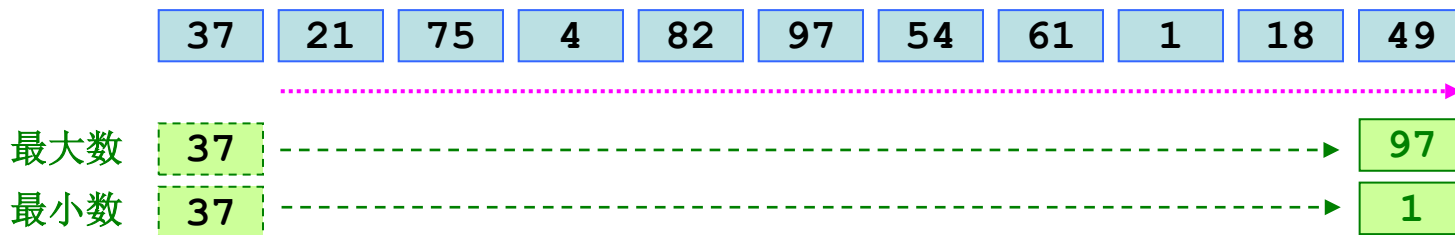
print('e =', e)
```



问题

✿ 任意输入11个数，找出最大数和最小数

分析与设计



✿ 数的范围无法确定

✿ 第一个数作为（此时的）最大数和最小数

✿ 将其余的数与最大数、最小数依次进行比较

✿ 每次根据比较的结果更新（此时的）最大数和最小数

✿ 用计数器控制的循环实现



找最大数

```
# maxValue的初值为第一个数
maxValue = int(input('输入一个整数: '))

count = 0
while count < 10:
    value = int(input('再输入一个整数: '))
    if maxValue < value:
        maxValue = value
    count += 1

print('the max value is', maxValue)
```

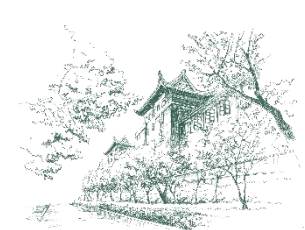
你来动手补上找
最小数的语句



武汉大学
WUHAN UNIVERSITY

06

实践指导



random模块简介

- Python标准库中的random模块是用于生成随机数的工具库。
- 它可以生成随机浮点数、整数、字符串，还可以帮助随机选择序列对象中的一个元素，或者打乱一组数据等。
- 以下是 random 模块中一些常用的函数：
 - (1) random(): 返回0和1之间的随机浮点数 n , $0.0 \leq n < 1.0$ 。
 - (2) uniform(a, b): 返回一个指定范围内的均匀分布的随机浮点数 N , $\min(a, b) \leq N \leq \max(a, b)$ 。生成的数值在数学上是均匀分布的，即区间内每个数出现的概率均等。
 - (3) randint(s, e): 返回区间 $[s, e]$ 中一个整数。
 - (4) choice(seq): 从序列seq中返回随机的元素。
 - (5) sample(seq, n): 从序列 seq 中选择 n 个随机且独立的元素。



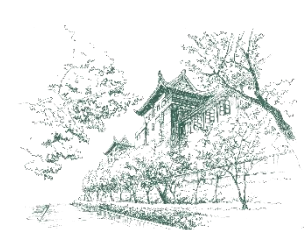
random模块简介



■ 在使用random模块的函数时，需要先导入random模块。

```
>>> import random
>>> random.random()
0.9745543203121101
>>> random.random()
0.8109950074660919
>>> random.randint(1, 10)
5
>>> random.randint(1, 10)
10
>>> random.randint(1, 10)
4
>>> random.uniform(5, 9)
8.679295115421406
>>> random.uniform(9, 5)
6.915750623937438
>>> random.uniform(2.3, 5)
4.681950876308827
```

```
>>> random.choice([1,2,3,4,5])
2
>>> random.choice([1,2,3,4,5])
5
>>> random.sample(['apple', 'pear', 'orange'], 2)
['pear', 'apple']
>>> random.sample(['apple', 'pear', 'orange'], 2)
['pear', 'orange']
```



应用案例1：猜数游戏程序（基础版）



武汉大学
WUHAN UNIVERSITY

■ 编写一个猜数游戏程序：随机产生一个1至100范围内的数，提示玩家来猜这个数，直到猜对为止。

```
import random

# 产生一个随机数
secret_number = random.randint(1, 100)

# 提示玩家去猜这个数
print("有一个1~100范围内的神秘数，请你来猜。")
# 猜数循环
while True:
    guessed_number = int(input("输入你猜的数: "))
    if guessed_number == secret_number:
        print("猜中了。你太牛了!")
        break
    print("很遗憾，没猜中，继续猜。")
# 游戏结束、
print("再见")
```




应用案例1：猜数游戏程序（改进思路）



武汉大学
WUHAN UNIVERSITY

■ 提高可玩性

✿ 如果用户猜的不对，给用户更多的提示

- 例如“你猜的数太大了”

✿ 提高趣味性

- 统计用户猜的次数，并给出评价
- 限制最大猜测次数

■ 增强功能

✿ 一次游戏结束后，询问用户是否再玩一次



应用案例1：猜数游戏程序（改进版）



```
"""改进后的猜数游戏程序"""
import random

# 产生一个随机数
secret_number = random.randint(1, 100)

# 提示玩家去猜这个数
print("有一个1~100范围内的神秘数，请你来猜。")

# 猜数循环
while True:
    guessed_number = int(input("输入你猜的数: "))
    if guessed_number == secret_number:
        print("猜中了。你太牛了!")
        break
    elif guessed_number < secret_number:
        print("很遗憾，没猜中。试试猜个更大的数。")
    else:
        print("很遗憾，没猜中。试试猜个更小的数。")

# 游戏结束
print("再见")
```



应用案例2：用蒙特卡洛方法估算圆周率



武汉大学
WUHAN UNIVERSITY

■ 蒙特卡洛方法是一种基于概率的数值计算方法

✱ 这个方法不是特指一种技术，而是一类基于概率和统计方法的统称。

✱ 基本思想是利用随机数（或更常见的伪随机数）来解决许多计算问题。

■ 蒙特卡洛方法可以用来估算圆周率，步骤如下：

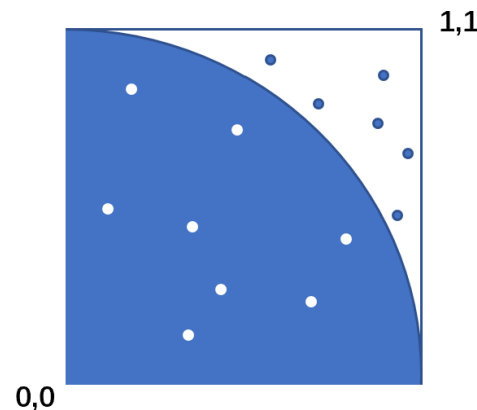
(1) 在一个边长为1的正方形内部画一个四分之一圆，其半径为1。

(2) 如图所示，随机在正方形内抛撒点，记录总的抛点数N和落在四分之一圆内的点数M。

(3) 根据随机点落在四分之一圆内的概率等于四分之一圆与正方形的面积之比，可以得到公式：

$$\begin{aligned} M / N &= \text{四分之一圆的面积} / \text{正方形的面积} \\ &= (\pi \cdot 1^2 / 4) / 1^2 = \pi / 4 \end{aligned}$$

(4) 从而可以估计出 $\pi = 4 \cdot M / N$ 。





应用案例2：用蒙特卡洛方法估算圆周率

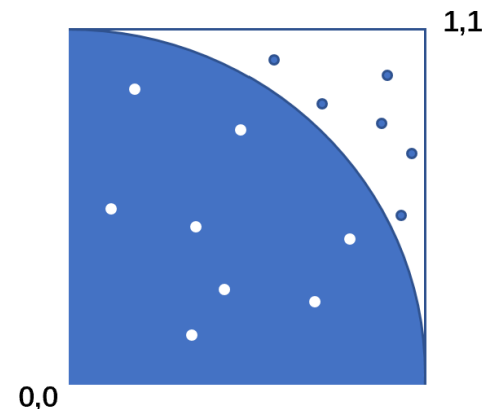


```
import random

# 输入抛撒的总点数
num_points = int(input("输入打算抛撒的总点数: "))
count = 0 # 已抛撒的点数
points_inside_circle = 0 # 在圆内的点数

# 随机抛撒点，统计落在圆内的点数
while count < num_points:
    # 产生一个随机点的横纵坐标值 (0~1)
    x = random.uniform(0, 1)
    y = random.uniform(0, 1)
    # 计算随机点与原点的距离
    distance = x ** 2 + y ** 2
    # 判断点是否在圆内
    if distance < 1:
        points_inside_circle += 1
    count += 1

# 估算圆周率
pi = 4 * points_inside_circle / num_points
print(f"估算的圆周率等于 {pi}")
```



执行程序，分别抛撒10000、100000、1000000个点，观察估算的圆周率。



应用案例3：投篮轨迹模拟



■ 设计思路

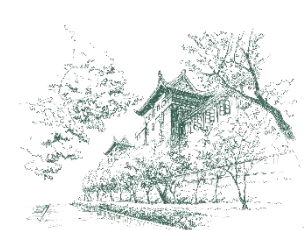
- ✿ 若要模拟篮球被投出后的运动轨迹，需要每隔一个时段，计算出篮球的位置。
- ✿ 计算方法采用上一章投篮模拟部分的投篮轨迹计算公式，根据投篮角度和出手速度，计算出每一时刻篮球的坐标值。
- ✿ 在计算的同时，绘制篮球的位置，就得到了一条投篮轨迹线。



应用案例3：投篮轨迹模拟

算法设计

算法名称	投篮轨迹实时绘制算法
输入参数	(1) 初始参数 - 篮圈高度 hoop_height (标准篮圈高度 3.05m) - 投篮距离 distance - 投篮高度 robot_height - 投篮角度 θ - 出手速度 v_0 (2) 环境参数 - 重力加速度 $g = 9.81 \text{ m/s}^2$ - 时间步长 Δt (默认 0.02s) - 绘制误差容限 ϵ (默认 0.1)
输出结果	(1) 实时绘制轨迹点 (逐点显示) (2) 投篮命中结果
核心公式	(1) 水平速度分量: $v_x = v_0 \cdot \cos(\theta)$ (2) 垂直速度分量: $v_y = v_0 \cdot \sin(\theta)$ (3) 轨迹方程: - $x(t) = x_0 + v_x \cdot t$ - $y(t) = y_0 + v_y \cdot t - \frac{1}{2}gt^2$ 其中, (x_0, y_0)是投篮的出手点坐标, 即 $(0, \text{robot_height})$。



应用案例3：投篮轨迹模拟



■ 算法设计

算法步骤	步 骤 说 明
1. 初始化	设置时间 $t=0$ ，命中标识 $hit=False$ 。
2. 轨迹循环计算	While True: 计算篮球的当前位置 $(x(t), y(t))$ 立即绘制点 检查篮球是否落地，如果是，则退出循环 检查是否命中篮圈，如果是，则标记 $hit=True$ ，并退出循环 $t = t + \Delta t$
3. 落地检测	篮球落地 ($y(t)<0$)
4. 命中检测	投篮命中 $ x(t)-x_{target} <\epsilon$ 且 $ y(t)-y_{target} <\epsilon$ ，其中 (x_{target}, y_{target}) 是篮圈中心的坐标，即 $(distance, hoop_height)$ 。
备 注	性能优化：若绘制延迟过高，可适当增大 Δt 或降低绘制频率。



应用案例3：投篮轨迹模拟



程序代码

```
import math
import turtle

# 重力加速度
g = 9.81

# 篮圈高度
hoop_height = 3.05 # 标准篮圈高度为3.05米

# 输入参数
distance = float(input("请输入投篮距离（米）：")) # 标准罚球线距离篮圈4.6米
robot_height = float(input("请输入投篮高度（米）："))
theta = float(input("请输入投篮角度（度）："))
v0 = float(input("请输入出手速度（米/秒）："))

# 计算篮圈高度与投篮高度的差值
h = hoop_height - robot_height

# 将角度转换为弧度
theta_rad = math.radians(theta)
```



应用案例3：投篮轨迹模拟



程序代码

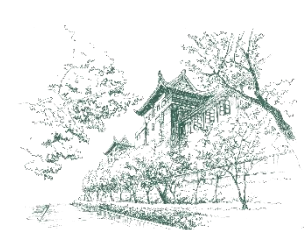
```
# 初始化 turtle
screen = turtle.Screen()
screen.setup(width=800, height=600)
screen.title("投篮轨迹模拟")

# 设置自定义坐标系
screen.setworldcoordinates(
    0, 0, # 左下角
    distance * 1.2, max(h, v0 ** 2 / (2 * g)) * 1.2 + robot_height) # 右上角

pen = turtle.Turtle()
pen.speed(0)
pen.penup()

# 绘制篮圈
pen.goto(distance, hoop_height - 0.225)
pen.pendown()
pen.color("purple")
pen.circle(0.225) # 篮圈半径约 0.225 米
pen.penup()

# 绘制出手点
pen.goto(0, robot_height)
pen.pendown()
pen.color("red")
pen.dot(10) # 绘制出手点
pen.color("black")
```



应用案例3：投篮轨迹模拟



程序代码

```
# 计算和绘制投篮轨迹
t = 0
dt = 0.02 # 时间步长
hit = False
while True:
    # 计算t时刻的篮球位置
    x = v0 * math.cos(theta_rad) * t
    y = v0 * math.sin(theta_rad) * t - 0.5 * g * t ** 2 + robot_height

    # 如果篮球落地或超出屏幕范围，停止计算和绘制
    if y < 0 or x > distance * 1.2:
        break

    # 绘制点
    pen.goto(x, y)
    pen.pendown()
    pen.dot(5) # 绘制篮球轨迹点

    # 判断是否命中篮圈，误差容限设为 0.1（篮球半径约 0.123 米）
    if abs(x - distance) < 0.1 and abs(y - hoop_height) < 0.1:
        hit = True
        # 命中，则停止计算和绘制
        break

t += dt
```



应用案例3：投篮轨迹模拟



程序代码

```
# 输出和绘制结果提示
if hit:
    print("投篮命中！")
    pen.penup()
    pen.goto(distance, hoop_height + 0.3)
    pen.pendown()
    pen.color("red")
    pen.write("命中！", align="left", font=("Arial", 16, "bold"))
else:
    print("投篮未命中。")
    pen.penup()
    pen.goto(x, y - 0.2)
    pen.pendown()
    pen.color("blue")
    pen.write("未命中！", align="right", font=("Arial", 16, "bold"))

# 结束
pen.hideturtle()
screen.mainloop()
```

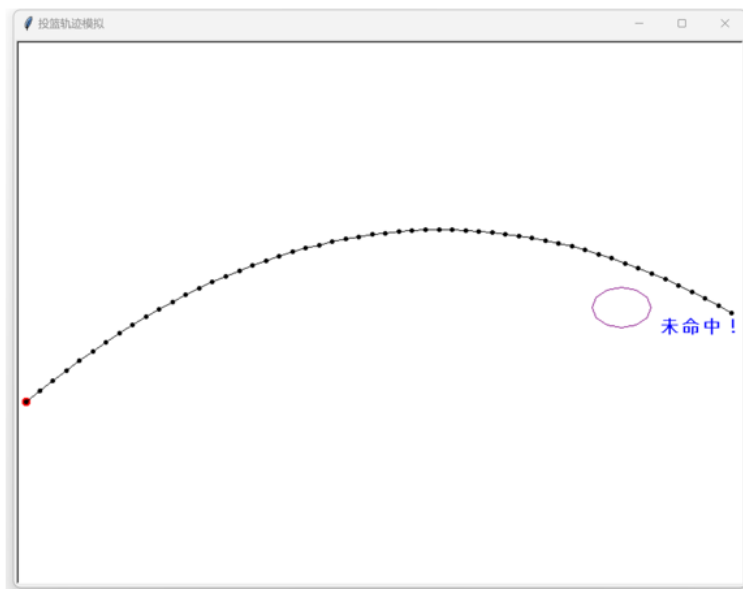


应用案例3：投篮轨迹模拟

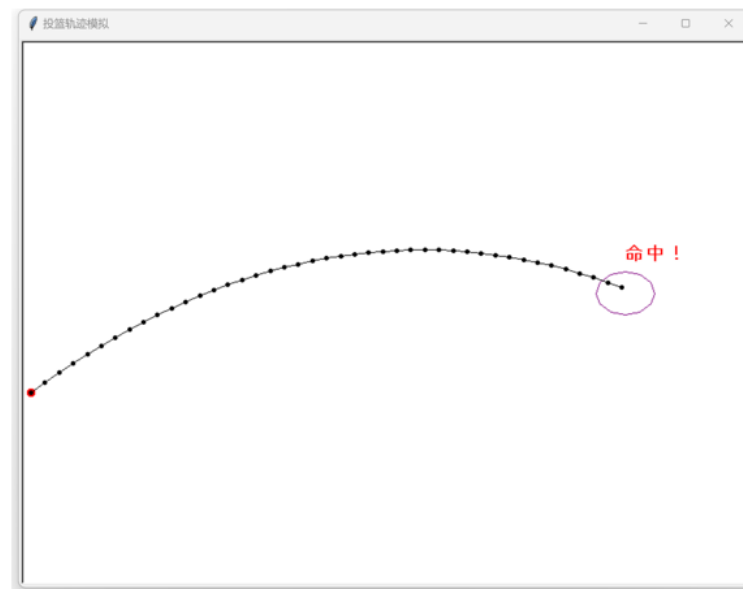


武汉大学
WUHAN UNIVERSITY

■ 执行效果



请输入投篮距离 (米) : 4.6
请输入投篮高度 (米) : 2
请输入投篮角度 (度) : 50
请输入出手速度 (米/秒) : 8
投篮未命中。



请输入投篮距离 (米) : 4.6
请输入投篮高度 (米) : 2
请输入投篮角度 (度) : 45
请输入出手速度 (米/秒) : 7.7
投篮命中!



武汉大学
WUHAN UNIVERSITY

07

程序错误与调试程序



程序的错误



■ 程序的错误（也称为异常）包括语法错误、运行时错误和逻辑错误。

■ 语法错误

✱ 指源代码中不符合Python语法规则的错误，这些错误导致Python解释器在编译阶段无法将源代码转换为字节码，因此也称之为编译错误。

```
print("Good Luck!"  
print("你今天的幸运随机数是: ", random.choice(range(10)))
```

```
File "E:\Course\syntax_error.py", line 1  
    print("Good Luck!"  
        ^  
SyntaxError: '(' was never closed
```




运行时错误

- ✱ 在程序中没有导入相关的模块（例如，`import random`）时，解释器将在运行时抛出 **NameError** 错误信息。
- ✱ 程序中包括零除运算，解释器将在运行时抛出 **ZeroDivisionError** 错误信息。
- ✱ 程序中试图打开不存在的文件，解释器将在运行时抛出 **FileNotFoundError** 错误信息。
- ✱

```
print("Good Luck!")  
print("你今天的幸运随机数是: ", random.choice(range(10)))
```

```
Good Luck!  
Traceback (most recent call last):  
  File "E:\Course\syntax_error.py", line 2, in <module>  
    print("你今天的幸运随机数是: ", random.choice(range(10)))  
                                ^^^^^^  
NameError: name 'random' is not defined
```



程序的错误



逻辑错误

- ✱ 程序可以执行（程序运行本身不报错），但执行结果不正确。
- ✱ 对于逻辑错误，Python解释器无能为力，需要读者根据结果来调试判断。

```
import math

a=1; b=2; c=1

x1 = -b + math.sqrt(b*b-4*a*c)/2*a    #公式有误，故结果不正确
x2 = -b - math.sqrt(b*b-4*a*c)/2*a    #公式有误，故结果不正确

print(x1, x2)    # 正确答案是：-1.0 -1.0
```

-2.0 -2.0

程序计算一元二次方程 $ax^2+bx+c=0$ 的两个根： $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ 。



语法错误的调试

```
>>> if 1 > 2      # 报错。 SyntaxError: invalid syntax
>>> Print('abc')  # 报错。 NameError: name 'Print' is not defined
>>> x             # 报错。 NameError: name 'x' is not defined
```

根据Python解释器抛出的异常，分别判断产生异常的原因：

- 第一条语句产生的SyntaxError表示语法错误（if复合语句后没有冒号）
- 第二条语句产生的NameError表示名称错误（拼写错误，应该为print）
- 第三条语句产生的NameError表示名称错误（未定义变量）



程序的基本调试方法



运行时错误的调试

```
>>> f = open('abc.txt') # 文件或者目录不存在: FileNotFoundError
>>> a=1;b=0
>>> c=a/b # 零除溢出: ZeroDivisionError
>>> 123 + 'abc' # TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

根据Python解释器运行时抛出的异常，分别判断产生异常的原因：

- 第一条语句产生的FileNotFoundError表示打开不存在的文件
- 第三条语句产生的ZeroDivisionError表示零除错误
- 第四条语句产生的TypeError表示不同类型对象值不能相加



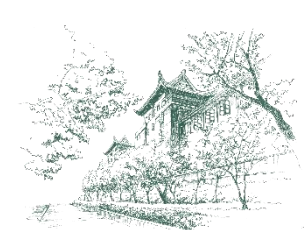
程序的基本调试方法



武汉大学
WUHAN UNIVERSITY

■ 逻辑错误的调试

- ✱ 断点跟踪、输出信息等方法。
- ✱ 有的集成开发环境（IDE）可以设置断点，并查看变量等。
- ✱ 通过print语句输出程序运行过程中变量值（跟踪信息），是观察和调试程序运行逻辑正确性的有效方法。



程序的基本调试方法



逻辑错误的调试

✿ 示例：下面程序的功能为找出 1 到 100 之间的所有质数，但程序有bug

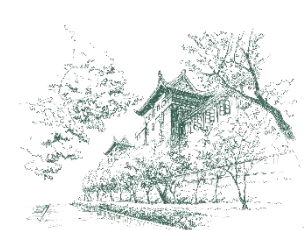
```
num = 2  # 从 2 开始检查

is_prime = True  # 假设当前数是质数，这行代码的位置有问题
while num <= 100:  # 检查 2 到 100 之间的数
    i = 2  # 内层循环变量，从 2 开始检查因数
    while i <= int(num**0.5):  # 只需检查到 sqrt(num)
        if num % i == 0:
            is_prime = False  # 如果有因数，则不是质数
            break
        i += 1  # 递增内层循环变量

    if is_prime:
        print(num, "是质数")

    num += 1  # 递增外层循环变量
```

2 是质数
3 是质数



程序的基本调试方法



武汉大学
WUHAN UNIVERSITY

逻辑错误的调试

添加print语句输出循环中相关变量的值

```
num = 2  # 从 2 开始检查

is_prime = True  # 假设当前数是质数，这行代码的位置有问题
while num <= 100:  # 检查 2 到 100 之间的数
    i = 2  # 内层循环变量，从 2 开始检查因数
    while i <= int(num**0.5):  # 只需检查到 sqrt(num)
        if num % i == 0:
            is_prime = False  # 如果有因数，则不是质数
            break
        i += 1  # 递增内层循环变量
```

`print("num:", num, "is_prime:", is_prime)` # 添加一行print语句，输出相关变量的值，观察循环执行过程

```
if is_prime:
    print(num, "是质数")
```

```
num += 1  # 递增外层循环变量
```

```
num: 2 is_prime: True
2 是质数
num: 3 is_prime: True
3 是质数
num: 4 is_prime: False
num: 5 is_prime: False
num: 6 is_prime: False
num: 7 is_prime: False
num: 8 is_prime: False
num: 9 is_prime: False
...
```



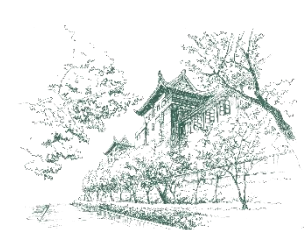

程序的基本调试方法



武汉大学
WUHAN UNIVERSITY

■ 使用调试器调试Python程序

- ✿ Python标准库的调试器pdb提供了基本的调试功能：设置断点、查看变量等。
- ✿ 集成开发环境（IDE，例如IDLE、Spyder、VS Code、PyCharm）则提供了更直观方便的调试器。



小结



■ 算法的基本概念

■ 关系运算和关系表达式

✱ $>$ 、 $<$ 、 $>=$ 、 $<=$ 、 $<>$ 、 $==$

■ 逻辑运算和逻辑表达式

✱ and 、 or 、 not

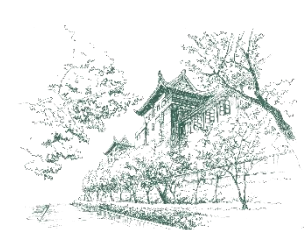
■ 选择结构

✱ If

✱ if-else

✱ if-elif-...-else

✱ match-case



小结



■ 循环结构

- ✿ while

- ✿ while-else

- ✿ break和continue语句

■ 实践指导

- ✿ 用标准库random模块生成随机数

- ✿ 用蒙特卡洛方法估算圆周率

- ✿ 用循环计算投篮轨迹

■ 编程难免会遇到程序错误，掌握基本的程序调试方法，可以提高编程效率。



武汉大学

WUHAN UNIVERSITY

谢谢

THANK YOU

